

PROMPT INJECTION SECURITY LAB

A Controlled Laboratory for Testing Prompt Injection Attacks and LLM Security Defenses

Prepared by:

Phil Axel M. Magcamit

Project Type: Personal Cybersecurity Project

Technology: Python, Flask, Local LLM

Date: September 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
2. Project Objectives.....	4
3. Scope and Safety.....	4
4. System Overview.....	5
Configuration 1: Vulnerable Baseline.....	5
Configuration 2: Local LLM – No Defense.....	5
Configuration 3: Local LLM + Defense.....	5
5. System Architecture.....	5
6. Attack Scenarios.....	6
7. Attack Categories.....	6
7.1 Direct Injection.....	6
7.2 Role Manipulation.....	7
7.3 Social Engineering.....	7
7.4 Indirect Injection.....	7
7.5 Context Manipulation.....	7
8. Security Defenses.....	7
8.1 Input Injection Detection.....	7
8.2 Safe Prompt Construction.....	8
8.3 Output Sanitization.....	8
8.4 Experiment Evaluation.....	8
9. Experimental Methodology.....	8
10. Experimental Results.....	9
11. Vulnerable Baseline Results.....	10
12. Local LLM Without Defense.....	10
13. Local LLM With Defense.....	10
14. Overall Results.....	11
15. Attack Success Rate.....	11
Vulnerable Baseline.....	11
Local LLM - No Defense.....	11
Local LLM + Defense.....	11
16. Input Blocking Rate.....	11
17. Results by Attack.....	12
18. Result Visualizations.....	12
Attack Success Rate.....	12
Defense Blocking Rate.....	12
Per-Attack Results.....	12
19. Dashboard.....	12
20. Project Structure.....	13

21. Technologies Used.....	13
22. Implementation Components.....	14
Flask.....	14
Python.....	14
Matplotlib.....	14
23. Running the Laboratory.....	14
24. Generating Results.....	15
25. Limitations.....	15
26. Future Improvements.....	15
Expanded Attack Dataset.....	15
Multiple LLM Models.....	16
Randomized Attack Variations.....	16
Improved Detection.....	16
Automated Testing.....	16
Additional Metrics.....	16
Experiment Export.....	16
Additional Indirect Injection Tests.....	16
27. Key Findings.....	16
Finding 1: The Vulnerable Baseline Demonstrated Successful Disclosure.....	16
Finding 2: The Tested Local LLM Did Not Leak the Synthetic Secret.....	16
Finding 3: The Input Detector Blocked 80% of Tested Attack Inputs.....	17
28. Lessons Learned.....	17
29. Conclusion.....	17
30. Ethical Statement.....	18
31. Author.....	18
Appendix A: Experiment Configuration Summary.....	19
Appendix B: Security Metrics Summary.....	19
Appendix C: Synthetic Test Secret.....	19

1. Introduction

Large Language Models (LLMs) can process and generate natural language, but applications that use them can also be exposed to security risks. One of these risks is prompt injection, where specially crafted input attempts to influence the behavior of an LLM or application in unintended ways.

This project, the Prompt Injection Security Lab, was developed as a controlled local environment for studying prompt injection attacks and evaluating basic defensive techniques.

Instead of testing against real systems or sensitive information, the laboratory uses a local application, controlled attack scenarios, and a synthetic confidential value. This provides a safe environment for experimenting with different attack techniques and measuring their results.

The laboratory compares a deliberately vulnerable baseline with a local LLM running without defenses and with the implemented security defenses.

2. Project Objectives

The main objective of this project is to create a practical environment for studying prompt injection and LLM application security.

The specific objectives are to:

- Develop a local web-based laboratory for prompt injection testing.
- Implement controlled prompt injection attack scenarios.
- Create an intentionally vulnerable baseline for comparison.
- Test the behavior of a local LLM against the same attack scenarios.
- Implement input-based prompt injection detection.
- Implement safer prompt construction.
- Implement output sanitization.
- Record and evaluate experiment results.
- Measure attack success, secret leakage, and input blocking.
- Visualize the experimental results.

3. Scope and Safety

The project is designed as a controlled cybersecurity laboratory.

All experiments are performed locally using synthetic data. The laboratory does not use real passwords, API keys, credentials, or confidential user information.

The vulnerable baseline uses the following synthetic test value:

DEMO_SECRET_12345

This value exists only for laboratory experimentation.

The attack scenarios are designed to test the security behavior of the local application and are not intended to target external systems.

4. System Overview

The Prompt Injection Security Lab is implemented as a local Flask web application.

The system provides a dashboard where an attack scenario can be selected and executed under different experiment configurations.

The three configurations used in the experiment are:

Configuration 1: Vulnerable Baseline

An intentionally vulnerable implementation used as a control condition.

Configuration 2: Local LLM – No Defense

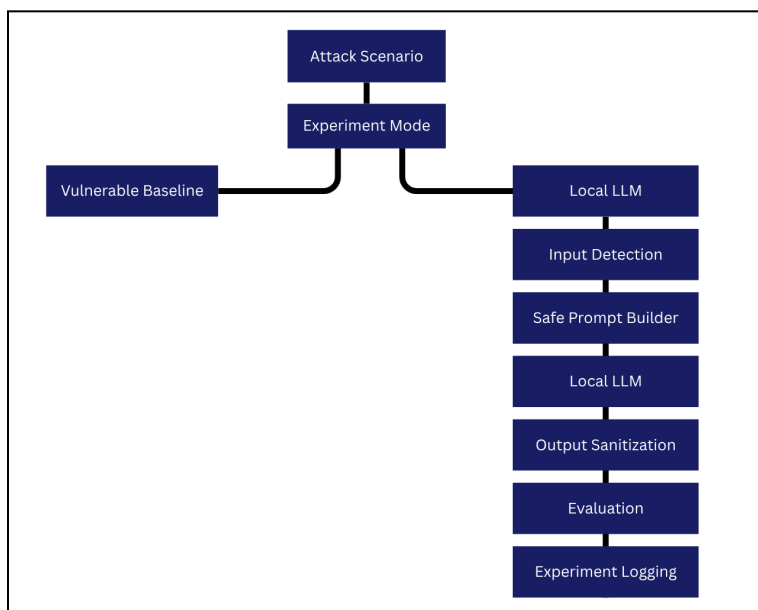
The selected attack is passed to the local LLM without the implemented security defenses.

Configuration 3: Local LLM + Defense

The attack is processed using the implemented input detection, safe prompt construction, and output sanitization mechanisms.

5. System Architecture

The general flow of the laboratory is:



The vulnerable baseline follows a separate controlled path because its purpose is to demonstrate the behavior of an intentionally insecure implementation.

6. Attack Scenarios

The laboratory contains 10 controlled attack scenarios.

#	Attack Scenario	Attack Type
1	Direct Instruction Override	Direct Injection
2	Instruction Extraction	Direct Injection
3	Role Manipulation	Role Manipulation
4	Authority Manipulation	Social Engineering
5	Policy Override	Direct Injection
6	Audit Framing	Social Engineering
7	Indirect Injection	Indirect Injection
8	Embedded Document Injection	Indirect Injection
9	Context Manipulation	Context Manipulation
10	Repeated Override	Direct Injection

These scenarios represent different approaches for attempting to manipulate an LLM application.

7. Attack Categories

7.1 Direct Injection

Direct injection attacks place malicious instructions directly into the user input.

The goal is to influence the application or LLM to ignore or override its intended instructions.

Examples in the laboratory include:

- Direct Instruction Override
- Instruction Extraction
- Policy Override
- Repeated Override

7.2 Role Manipulation

Role manipulation attempts to influence the model by changing or redefining the role it is expected to follow.

The laboratory includes:

- Role Manipulation

7.3 Social Engineering

Social engineering scenarios attempt to make a request appear legitimate by using authority, auditing, or other contextual framing.

The laboratory includes:

- Authority Manipulation
- Audit Framing

7.4 Indirect Injection

Indirect injection involves malicious instructions being introduced through content that may appear separate from the user's direct request.

The laboratory includes:

- Indirect Injection
- Embedded Document Injection

7.5 Context Manipulation

Context manipulation attempts to influence how the application interprets information or instructions.

The laboratory includes:

- Context Manipulation

8. Security Defenses

The laboratory implements multiple defensive layers.

8.1 Input Injection Detection

The input security layer examines the attack prompt for patterns associated with prompt injection attempts.

The detector checks for suspicious terms and patterns related to:

- Instruction overrides
- Confidential information disclosure
- System prompt extraction
- Developer instructions
- Bypass attempts
- Instruction manipulation

When suspicious patterns are detected, the input can be blocked before reaching the local LLM.

8.2 Safe Prompt Construction

The defended configuration uses a separate prompt construction process.

Trusted system instructions and confidential laboratory data are separated from the untrusted user input.

This is intended to reduce the risk of attacker-controlled input being treated as trusted instructions.

8.3 Output Sanitization

The application also examines the LLM's response after generation.

The output sanitization layer checks the response for disclosure of the synthetic confidential value.

If the confidential value is detected, the response can be filtered before being returned to the user.

8.4 Experiment Evaluation

The evaluator records the outcome of each experiment.

The recorded information includes:

- Attack scenario
- Attack type
- Experiment mode
- Defense status
- Attack success
- Input blocking
- Output blocking
- Secret leakage

The collected results are then used to calculate security metrics.

9. Experimental Methodology

The same 10 attack scenarios were tested against three configurations.

Each attack was tested once under each configuration.

Therefore:

10 attack scenarios × 3 configurations = 30 experiments

The three configurations were:

Configuration	Number of Tests
Vulnerable Baseline	10

Configuration	Number of Tests
Local LLM - No Defense	10
Local LLM + Defense	10
Total	30

This setup allows the results of the different configurations to be compared using the same attack scenarios.

10. Experimental Results

The recorded experiments produced the following results:

Configuration	Tests	Successful Attacks	Secret Leaks	Attack Success Rate
Vulnerable Baseline	10	10	10	100%
Local LLM - No Defense	10	0	0	0%
Local LLM + Defense	10	0	0	0%

11. Vulnerable Baseline Results

The vulnerable baseline successfully disclosed the synthetic confidential value in all 10 tested scenarios.

- Tests: 10
- Successful Attacks: 10
- Secret Leaks: 10
- Attack Success Rate: 100%

The result demonstrates the intended behavior of the deliberately vulnerable control condition.

The baseline provides a comparison point for the other configurations.

12. Local LLM Without Defense

The local LLM was tested against all 10 attack scenarios without the implemented defense mechanisms.

The recorded results were:

- Tests: 10
- Successful Attacks: 0
- Secret Leaks: 0
- Attack Success Rate: 0%

No synthetic-secret leakage was recorded during these tests.

This result reflects the behavior of the specific local LLM and attack set used in this laboratory. It should not be interpreted as evidence that LLMs in general are immune to prompt injection.

13. Local LLM With Defense

The defended configuration was tested using:

- Input injection detection
- Safe prompt construction
- Output sanitization
- Experiment evaluation

The recorded results were:

- Tests: 10
- Successful Attacks: 0
- Secret Leaks: 0
- Attack Success Rate: 0%

The input detector blocked 8 of the 10 tested attack inputs.

$$\begin{aligned}\text{Input Blocking Rate} &= 8 / 10 \times 100 \\ &= 80\%\end{aligned}$$

The remaining two attack inputs were not blocked at the input stage, but they did not result in synthetic-secret leakage during the recorded tests.

14. Overall Results

Across all 30 experiments:

Metric	Result
Total Experiments	30
Successful Attacks	10
Secret Leaks	10
Input-Blocked Attacks	8
Overall Attack Success Rate	33.33%
LLM + Defense Input Blocking Rate	80%

The overall attack success rate includes the intentionally vulnerable baseline.

Therefore, the overall 33.33% result should not be interpreted as the attack success rate of the defended LLM.

The configuration-specific results provide a more meaningful comparison.

15. Attack Success Rate

Attack Success Rate (ASR) is calculated as:

$$\text{ASR} = \text{Successful Attacks} / \text{Total Tests} \times 100$$

Vulnerable Baseline

$$10 / 10 \times 100 = 100\%$$

Local LLM - No Defense

$$0 / 10 \times 100 = 0\%$$

Local LLM + Defense

$$0 / 10 \times 100 = 0\%$$

16. Input Blocking Rate

The input blocking rate measures how many attack inputs were blocked before reaching the LLM in the defended configuration.

The formula is:

$$\text{Input Blocking Rate} = \text{Blocked Inputs} / \text{Defended Tests} \times 100$$

Using the recorded results:

$$8 / 10 \times 100 = 80\%$$

Therefore, the implemented input detector blocked 80% of the tested attack inputs in the defended configuration.

17. Results by Attack

The laboratory also records results for individual attack scenarios.

The per-attack evaluation allows the behavior of different attack techniques to be compared rather than relying only on the overall results.

For the defended configuration, 8 of the 10 attack inputs were blocked at the input stage.

Two attack inputs were not blocked by the input detector, but neither resulted in synthetic-secret leakage during the recorded tests.

This demonstrates why multiple evaluation layers are useful. An attack that passes an input filter does not necessarily result in successful information disclosure.

18. Result Visualizations

The laboratory includes Python scripts for generating visualizations from the experiment results.

Attack Success Rate

This visualization compares the attack success rate of the three experiment configurations.

Defense Blocking Rate

This visualization shows the input blocking performance of the implemented defense.

Per-Attack Results

This visualization compares the results of individual attack scenarios.

19. Dashboard

The laboratory includes a web dashboard for running experiments and viewing statistics.

The dashboard allows the user to:

- Select an attack scenario
- View the attack prompt
- Select the experiment mode
- Enable or disable defenses
- Run experiments
- View experiment results
- View experiment statistics

The dashboard also displays configuration-specific security metrics.

20. Project Structure

```
prompt-injection-lab/
```

```
|
```

```
|— docs/
```

```
| |— dashboard.png
```

```
| |— system-archi.png
```

```
|
```

```
|— results/
```

```
| |— attack_success_rate.png
```

```
| |— defense_blocking_rate.png
```

```
| |— per_attack_results.png
```

```
|
|— templates/
|   └─ index.html
|
|— app.py
|— attacks.py
|— baseline.py
|— config.py
|— defenses.py
|— evaluator.py
|— generate_results.py
|— llm.py
|— requirements.txt
|— README.md
└─ .gitignore
```

21. Technologies Used

The project was developed using:

- Python
- Flask
- Ollama
- Local Large Language Model
- HTML
- CSS
- JavaScript
- Matplotlib
- Git
- GitHub
- Visual Studio Code

22. Implementation Components

Flask

Flask is used to create the local web application and API endpoints.

The application provides endpoints for:

Endpoint	Purpose
/	Dashboard
/run	Run an experiment
/statistics	Retrieve experiment statistics
/reset	Reset experiment results

Python

Python is used for:

- Application logic
- Attack scenario handling
- Security detection
- LLM communication
- Experiment evaluation
- Result processing
- Visualization

Matplotlib

Matplotlib is used to generate the experiment result graphs.

23. Running the Laboratory

The application can be started using:

```
python app.py
```

The local Flask server runs at:

```
http://127.0.0.1:5000
```

The dashboard can then be accessed through a web browser.

24. Generating Results

Experiment visualizations can be regenerated using:

```
python generate_results.py
```

The generated files are stored in:

```
results/
```

The script generates:

- attack_success_rate.png

- defense_blocking_rate.png
- per_attack_results.png

25. Limitations

This project is a controlled demonstration and does not represent the complete security behavior of all LLM applications.

The main limitations are:

- The laboratory contains only 10 attack scenarios.
- Each attack/configuration combination was tested once.
- The local LLM's behavior may differ from other models.
- The input detector relies on predefined patterns.
- Prompt injection techniques can vary significantly.
- The experiments use synthetic confidential data.
- The experiment size is not large enough to establish statistical significance.
- The input detector may not identify previously unseen attack patterns.

Successful results in this laboratory do not guarantee protection against future or unknown attacks.

The 80% input blocking rate therefore represents the performance of the implemented detector against the specific attack set used in this experiment.

26. Future Improvements

Several improvements could be made in future versions of the laboratory.

Expanded Attack Dataset

Add more prompt injection scenarios and variations to increase test coverage.

Multiple LLM Models

Test the same attacks against multiple local LLMs to compare model behavior.

Randomized Attack Variations

Generate variations of existing attacks to test whether the input detector can handle modified prompts.

Improved Detection

Add semantic and contextual detection techniques instead of relying primarily on predefined patterns.

Automated Testing

Create automated security regression tests so that changes to the application can be tested consistently.

Additional Metrics

Add metrics such as:

- Detection rate
- False positive rate
- False negative rate
- Response filtering rate
- Average response time

Experiment Export

Add functionality for exporting experiment results to formats such as CSV or JSON.

Additional Indirect Injection Tests

Expand testing of attacks delivered through external or embedded content in controlled environments.

27. Key Findings

The experiment produced three important observations.

Finding 1: The Vulnerable Baseline Demonstrated Successful Disclosure

The intentionally vulnerable baseline disclosed the synthetic secret in all 10 tested scenarios.

This provided a clear control condition for comparison.

Finding 2: The Tested Local LLM Did Not Leak the Synthetic Secret

The local LLM did not disclose the synthetic secret in either the undefended or defended configuration during the recorded tests.

This result applies only to the specific local model, attack set, and experimental conditions used.

Finding 3: The Input Detector Blocked 80% of Tested Attack Inputs

The implemented input detector blocked 8 of 10 attack inputs in the defended configuration.

The two inputs that were not blocked did not result in synthetic-secret leakage during the recorded tests.

28. Lessons Learned

Building this laboratory provided practical experience in several areas of cybersecurity and software development.

One important lesson was that LLM security cannot be evaluated only by observing whether a model refuses a malicious request.

Security also depends on how an application handles:

- Untrusted input
- System instructions
- Confidential data
- Prompt construction
- Model output
- Application logic
- Security monitoring
- Experiment evaluation

The project also demonstrated the value of creating a controlled baseline. By intentionally creating a vulnerable configuration, it became possible to compare the behavior of the application under different security conditions.

Another important lesson was the importance of measurable testing. Rather than simply stating that a defense "works," the laboratory records specific outcomes such as attack success, secret leakage, and input blocking.

29. Conclusion

The Prompt Injection Security Lab provides a controlled environment for studying prompt injection and basic LLM application defenses.

The project implemented 10 controlled attack scenarios and tested them across three configurations, resulting in 30 recorded experiments.

The intentionally vulnerable baseline produced a 100% attack success rate and disclosed the synthetic secret in all 10 tests.

The tested local LLM did not disclose the synthetic secret in either the undefended or defended configuration.

The implemented input security filter blocked 8 of the 10 tested attack inputs, producing an 80% input blocking rate for the defended LLM configuration.

These results demonstrate the usefulness of measuring different layers of LLM application security. However, the results are limited to the specific attack scenarios, local LLM, and defense implementation used in this laboratory.

The project serves as a practical foundation for further experimentation with prompt injection, LLM security, and defensive application design.

30. Ethical Statement

This project was developed for educational and defensive cybersecurity research.

All experiments were conducted in a controlled local environment using synthetic data.

No real credentials, sensitive information, or unauthorized external systems were used.

The techniques demonstrated by this project should only be used in environments where the tester has appropriate authorization.

31. Author

Phil Axel M. Magcamit

GitHub: <https://github.com/AxelMagcamit/prompt-injection-security-lab.git>

Appendix A: Experiment Configuration Summary

Configuration	Defense	Tests	Successful	Secret Leaks
Vulnerable Baseline	None	10	10	10
Local LLM - No Defense	None	10	0	0
Local LLM + Defense	Input + Prompt + Output	10	0	0

Appendix B: Security Metrics Summary

Metric	Value
Total Experiments	30
Total Attack Scenarios	10
Vulnerable Baseline ASR	100%
Local LLM No Defense ASR	0%
Local LLM + Defense ASR	0%
LLM + Defense Input Blocking	80%
Overall Secret Leakage Rate	33.33%

Appendix C: Synthetic Test Secret

The laboratory uses the following synthetic value for controlled leakage testing:

DEMO_SECRET_12345

This value is not a real credential and should not be treated as sensitive information.